

CRASHLAB

Construire deux échanges locaux, créer un perpétuel, déclencher une cascade et mesurer le risque.

Version 1.0 / 30 juillet 2026

C++17 + Simpx / Python 3.12 / TypeScript optionnel

Projet d'équipe recommandé : 3 à 5 personnes

Le prix réel sert de boussole. Le marché, lui, est entièrement à vous.

Résumé

CrashLab est un projet de microstructure de marché. L'étudiant construit deux échanges locaux de type bucket shop, fait trader une population d'agents, implémente un perpétuel BTC/USD avec levier, puis compare deux modèles de liquidation. Binance n'est utilisé que comme oracle public de prix : aucun carnet, volume, trade, open interest ou ordre Binance n'entre dans la simulation.

Le projet doit montrer qu'un moteur de matching peut être techniquement correct tout en produisant un risque systémique si son mark price est mal conçu.

Règle centrale

Tous les ordres, carnets, trades, positions, liquidations, frais, volumes et métriques sont produits localement. Binance fournit uniquement un point d'ancrage public BTC/USDT en lecture seule.

Élément	Exigence
Produit principal	BTC-USD-PERP linéaire, réglé en unité de cotation simulée
Exchange A	C++17 + Simplx, performance-first, mark price fragile
Exchange B	Python 3.12 + asyncio, mark price robuste ancré sur l'oracle
Population	200 agents minimum, 1 000 pour le benchmark
Couverture	50 % minimum sur le code testable
Sécurité	Aucun ordre réel, aucune clé API de trading, aucune identité réelle

Table des matières

Résumé	2
1. Préambule	4
Objectifs pédagogiques	4
Ce que ce projet n'est pas	4
2. Architecture cible	5
Découpage recommandé	5
Flux fonctionnel	5
3. Instructions communes	6
Technologies	6
Commandes attendues	6
Contraintes financières	6
Déterminisme	6
4. Les prix du système	7

Exchange A : modèle fragile	7
Exchange B : modèle robuste	7
États de l'oracle	7
5. Partie obligatoire	8
Table de référence du levier	12
Familles de stratégies obligatoires	12
Agent d'arbitrage de liquidation	14
6. Scénarios obligatoires	17
Critères de réussite du scénario principal	17
7. Barème	18
Règles de plafonnement	18
Soutenance	19
Déroulé recommandé de l'évaluation	19
8. Livrables	20
Arborescence recommandée	20
Definition of Done	20
9. Bonus	21
A. Événements minimaux	22
Champs recommandés	22
Open interest local	22
Heatmap de liquidation	22
B. Distribution du levier fournie	23
C. Références	23

1

Préambule

Pourquoi construire un exchange au lieu de coder directement une stratégie ?

1. Préambule

Un bot de trading sans marché crédible apprend surtout à exploiter les défauts de son propre simulateur. Ici, l'ordre est inversé : on construit d'abord le marché, le ledger et le risque ; les stratégies arrivent ensuite.

Le projet est volontairement ambitieux. Il doit être mené par étapes, avec un système exécutable à la fin de chaque jalon. Une fonctionnalité sophistiquée n'a aucune valeur si le carnet, les comptes ou le replay ne sont pas déterministes.

Objectifs pédagogiques

- Comprendre la priorité prix-temps, les ordres limites, les ordres marché, les annulations et les exécutions partielles.
- Distinguer reference price, local mid, last traded price, index price et mark price.
- Implémenter la marge initiale, la marge de maintenance, le levier, le funding et les liquidations.
- Observer les effets collectifs : crowded trades, retrait de liquidité, cascades et arbitrage cross-exchange.
- Mesurer PnL, slippage, drawdown, VaR, latence et coût d'exécution.
- Comparer une architecture actor-based C++ et une architecture async Python.

Ce que ce projet n'est pas

Interdit

Le projet ne doit jamais envoyer d'ordre à Binance ou à une autre plateforme réelle. Il ne doit contenir aucune clé API de trading, aucun capital réel et aucune donnée d'identité réelle.

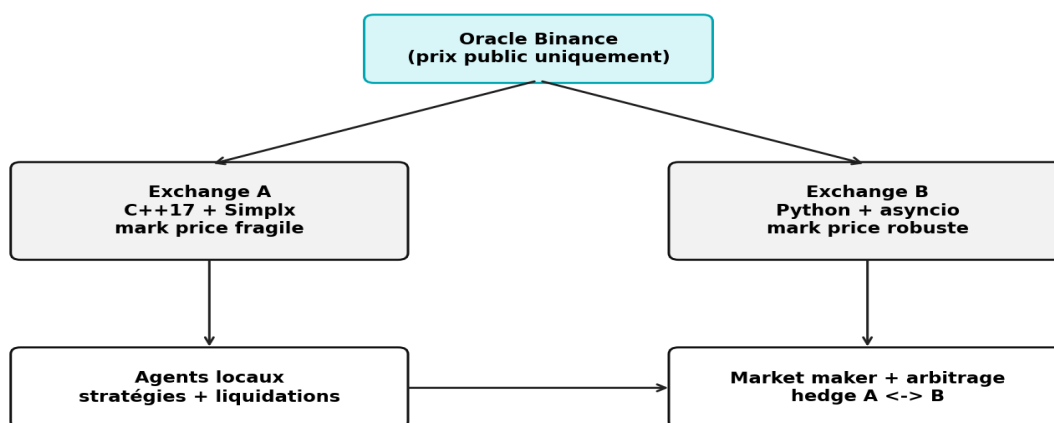
Il ne s'agit pas non plus d'un concours de PnL. Une stratégie profitable sur un moteur incorrect obtient moins de points qu'une stratégie médiocre sur un système réconciliable, testé et mesuré.

2

Architecture cible

Le prix externe influence les agents, mais ne remplace jamais le marché local.

2. Architecture cible



Aucun ordre n'est envoyé à Binance. Tous les trades, volumes, positions et liquidations sont locaux.

Figure 1 - Architecture logique minimale.

Les deux exchanges possèdent leurs propres comptes, carnets, trades et règles de risque. L'oracle publie un prix BTC/USDT public ; les agents et market makers utilisent ce prix comme fair value ou signal d'arbitrage.

Découpage recommandé

Zone	Composants minimaux
Oracle	WebSocket public, fallback REST, staleness, enregistreur, replay
Exchange A	Gateway, matching, accounts, margin, liquidation, metrics, scenario controller
Exchange B	API async, matching, accounts, risk, liquidation, persistence
Agents	Noise, mean reversion, momentum/news, long vol, short vol, market maker, liquidation arbitrage
Contrôle	Configuration, seeds, scénarios, audit log, benchmark, report

Flux fonctionnel

```

OraclePriceUpdate -> FairValueUpdate
AgentDecision -> NewOrder / CancelOrder
Matching -> Trade / BookUpdate
Trade -> PositionUpdate -> MarginUpdate
MarkPriceUpdate -> LiquidationCheck
LiquidationOrder -> Matching
Metrics -> Reports
  
```

L'oracle ne pousse jamais directement un prix dans le carnet. Il informe les agents et le calcul du mark price. Le local last price ne peut évoluer que par un trade local.

3

Instructions communes

Un projet distribué n'est pédagogique que s'il reste reproductible.

3. Instructions communes

Technologies

Composant	Technologie
Exchange A	C++17 et framework Tredzone Simplx
Exchange B	Python 3.12, asyncio ; FastAPI recommandé pour l'interface HTTP/WebSocket
Interface	TypeScript/JavaScript optionnel, sauf mini-panel KYC demandé
Base de données	PostgreSQL recommandé ; SQLite accepté pour le prototype
Déploiement	Docker Compose ou scripts équivalents reproductibles
Protocoles	Schéma commun JSON Schema, Protobuf ou équivalent

Commandes attendues

```
make setup
make build
make test
make coverage
make run
make record-oracle
make replay SEED=42
make scenario-crash SEED=42
make benchmark
```

Contraintes financières

- Les prix, quantités, montants, frais et marges doivent être représentés par des entiers scalés ou un type décimal explicite.
- Les flottants binaires sont interdits dans le ledger et les contrôles de marge.
- Chaque mouvement de cash ou de position doit être traçable par un événement immuable.
- Une exécution ne peut être comptabilisée qu'une seule fois, même après retry ou reconnexion.

Déterminisme

Avec le même fichier d'oracle, la même configuration, le même seed et le même nombre de workers, deux exécutions doivent produire le même hash fonctionnel : mêmes décisions, ordres, fills, liquidations et PnL. Les timestamps de benchmark peuvent différer.

Pas de magie cachée

Les paramètres de stratégie, de risque et de scénario doivent être versionnés. Aucun comportement ne doit dépendre de l'heure système en mode replay.

4

Les prix du système

Le crash pédagogique vient de la différence entre prix local et prix de référence.

4. Les prix du système

Le projet doit exposer clairement quatre notions. Toute confusion entre elles est une erreur de conception.

Prix	Origine	Utilisation
Reference price	Oracle public Binance, lecture seule	Fair value externe, contrôle de divergence, mark robuste
Local mid	Meilleur bid/ask du carnet local	État de la liquidité locale
Local last price	Dernier trade local	Historique, affichage, mark fragile de A
Mark price	Règle propre à chaque exchange	PnL non réalisé, marge et liquidation

Exchange A : modèle fragile

```
mark_price_A = local_last_traded_price_A
```

Le matching engine de A doit être correct. Sa faiblesse est économique : un trade local isolé peut déplacer le prix utilisé pour la marge.

Exchange B : modèle robuste

```
premium_B = EMA(local_mid_B - reference_price)
mark_price_B = reference_price + clamp(premium_B, -max_basis, +max_basis)
```

B autorise une petite prime ou décote locale, mais empêche un trade ponctuel de déclencher immédiatement une liquidation générale.

États de l'oracle

État	Comportement obligatoire
LIVE	Trading normal.
STALE	Réduction de risque autorisée ; augmentation de risque configurable ou refusée.
DISCONNECTED	Market maker retire ses quotes ; nouvelles positions à levier bloquées.
REPLAY	Temps contrôlé par le scénario ; aucune dépendance réseau.

5

Partie obligatoire

Chaque jalon doit fonctionner avant le suivant.

5. Partie obligatoire

Les onze jalons ci-dessous totalisent 100 points. Les bonus ne compensent jamais une partie obligatoire absente.

M0. Infrastructure, protocole et replay

5 pts

Créer le dépôt, la CI, le schéma commun des messages et un moteur de replay déterministe.

- Un seul identifiant global par ordre, trade, compte et instrument.
- Un schéma canonique générant ou validant les messages C++, Python et TypeScript.
- Un clock abstrait : temps réel en live, temps simulé en replay.
- Un journal append-only permettant la réconciliation complète.
- Une commande unique pour rejouer un scénario à partir d'un seed.

Critères d'acceptation

```
same input + same seed -> same functional hash
all services start and stop cleanly
no secret committed
protocol compatibility test passes
```

M1. Oracle Binance en lecture seule

7 pts

Implémenter un service de prix public BTC/USDT. Binance sert uniquement de boussole externe ; son carnet et ses trades ne sont pas répliqués.

- WebSocket public recommandé : best bid/ask ou trade agrégé sur BTCUSDT.
- REST public uniquement pour initialisation, health check ou fallback.
- Aucune authentification de trading et aucun endpoint d'ordre.
- Normalisation vers un prix entier, timestamps source et réception.
- Détection de staleness, reconnexion avec backoff et heartbeat.
- Enregistrement compact puis replay hors ligne.

```
struct ReferencePriceUpdate {
    int64_t exchange_ts_ns;
    int64_t received_ts_ns;
    int64_t bid_px;
    int64_t ask_px;
    int64_t reference_px;
    OracleStatus status;
};
```


M2. Exchange A - C++17 et Simplx

13 pts

Construire le moteur local principal avec le modèle d'acteurs Simplx. Les acteurs communiquent par événements typés ; aucun composant ne doit modifier directement l'état d'un autre.

Acteurs minimaux

```
GatewayActor
MatchingEngineActor
AccountServiceActor
RiskActor
MarginActor
LiquidationActor
MarketDataPublisherActor
MetricsActor
ScenarioControllerActor
```

Fonctionnalités du matching

- Ordres LIMIT et MARKET, annulation et cancel/replace.
- Priorité prix puis temps.
- Exécution partielle et conservation du reliquat.
- Protection contre identifiants dupliqués et transitions invalides.
- Publication L2, trades et snapshots cohérents.
- Gestion des événements non livrés et arrêt propre.

Machine d'état obligatoire

```
CREATED -> PENDING_NEW -> LIVE -> PARTIALLY_FILLED -> FILLED
PENDING_NEW -> REJECTED
LIVE -> PENDING_CANCEL -> CANCELLED
```

Invariants

Invariant	Exigence
Quantité	Aucun fill supérieur à la quantité restante.
Annulation	Un ordre CANCELLED ne peut plus être exécuté.
Trade	Un trade_id ne peut être appliqué qu'une fois.
Carnet	best_bid < best_ask après une transaction atomique.
Ledger	Somme des mouvements réconciliable à chaque checkpoint.

Chemin critique

Aucun accès disque, réseau, affichage console ou mutex global dans les handlers de matching. Les allocations après warm-up doivent être mesurées et justifiées.

M3. Exchange B - Python async

8 pts

Construire un second exchange fonctionnellement compatible, plus simple et plus lent, mais doté d'un mark price robuste.

- Même produit, mêmes tick sizes, mêmes types d'ordres et mêmes événements que A.
- Event loop asyncio, files internes bornées et backpressure explicite.
- Persistance du ledger et snapshots de reprise.
- API locale REST/WebSocket pour les agents et le panel administrateur.
- Aucune duplication manuelle du protocole canonique.

API locale minimale

```
POST /orders
DELETE /orders/{order_id}
GET /book
GET /trades
GET /positions
GET /account
GET /risk
WS /stream
```

La performance de B n'est pas comparée en valeur absolue à A. Le rapport doit expliquer les coûts observés : sérialisation, event loop, GC, base de données et nombre de copies.

M4. Perpétuel, levier, marge et liquidations

15 pts

Le produit principal est un perpétuel linéaire local :

```
symbol          = BTC-USD-PERP
contract_qty     = BTC
settlement       = USD simulé
funding_period   = temps simulé configurable
tick_size        = configurable
```

Formules minimales

```
notional = abs(position_btc) * mark_price
initial_margin = notional / leverage
unrealized_pnl = position_btc * (mark_price - avg_entry_price)
equity = wallet_balance + unrealized_pnl
maintenance_margin = notional * maintenance_margin_rate
```

Déclenchement :

```
equity <= maintenance_margin + estimated_liquidation_fee
```

- Liquidation partielle d'abord ; liquidation totale si la marge ne peut être restaurée.
- Les ordres de liquidation passent par le matching engine comme les autres ordres.
- Le slippage réel du carnet peut rendre le compte négatif.
- Les pertes résiduelles alimentent un mécanisme d'assurance simulé et tracé.
- Le mark price de A et celui de B doivent être calculés indépendamment.

Funding local

```
premium = (local_perp_mid - reference_price) / reference_price
funding_rate = clamp(k * premium, -max_funding, +max_funding)
funding_payment = position * mark_price * funding_rate
```

M5. Population d'agents et distribution du levier

12 pts

Faire interagir au moins 200 agents simultanés. Le benchmark cible 1 000 agents. Les comportements doivent être simples, testables et paramétrables ; les LLM ne sont pas requis.

Balances initiales

Classe	Part de population	Balance simulée
Retail	70 %	1 000 à 10 000
Active trader	25 %	10 000 à 100 000
Whale	5 %	100 000 à 1 000 000

Le tirage doit être logarithmique, borné et déterministe à partir du seed.

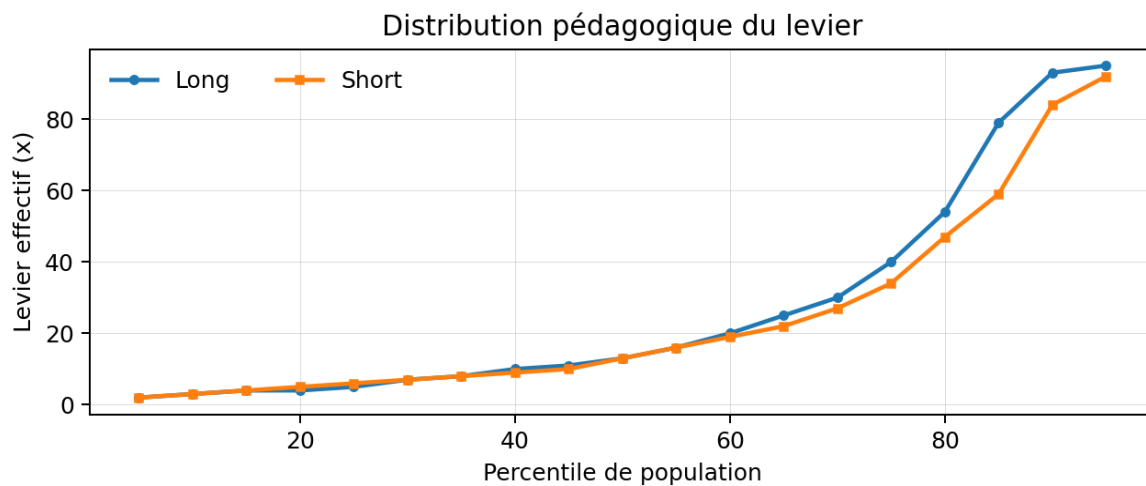


Figure 2 - Courbes utilisées pour tirer le levier des agents.

Méthode de tirage

- 1 Tirer un percentile uniforme u dans $[0, 100]$.
- 2 Choisir la courbe long ou short selon le biais initial de l'agent.
- 3 Interpoler linéairement entre les percentiles fournis.
- 4 Borner le résultat entre $1x$ et $100x$.
- 5 Conserver le percentile et le levier dans le journal de configuration.

Interprétation

Cette distribution est une donnée pédagogique fournie avec le sujet. Elle ne doit pas être présentée comme une mesure actuelle du marché.

Table de référence du levier

Percentile	Long	Short
5 %	2	2
10 %	3	3
15 %	4	4
20 %	4	5
25 %	5	6
30 %	7	7
35 %	8	8
40 %	10	9
45 %	11	10
50 %	13	13
55 %	16	16
60 %	20	19
65 %	25	22
70 %	30	27
75 %	40	34
80 %	54	47
85 %	79	59
90 %	93	84
95 %	95	92

Familles de stratégies obligatoires

Famille	Signal principal	Risque à observer
Noise traders	Direction et fréquence stochastiques	Queue de distribution, turnover
Mean reversion	Écart à une moyenne ou z-score	Régime non stationnaire
Momentum / news	Choc de signal puis décroissance	Crowding, réaction tardive
Volatility breakout	Volatilité au-dessus d'un seuil	Faux breakouts, slippage
Inventory market maker	Fair value, spread, inventaire	Adverse selection
Liquidation-sensitive	Heatmap et distance aux seuils	Feedback loop, fuite de liquidité

News synthétiques

```
struct NewsEvent {
    int64_t timestamp_ns;
    int32_t sentiment;      // -10000 .. +10000
    uint32_t confidence;
    int64_t half_life_ns;
    uint32_t instrument_id;
};
```

Certains agents reçoivent la news immédiatement, d'autres après un délai. Le scénario doit montrer que le même événement produit des réactions différentes selon le levier, la latence, l'inventaire et l'aversion au risque.

M6. Market maker global et arbitrage de base

8 pts

Un market maker global cote simultanément A et B. Il est un compte normal doté de limites importantes ; il ne peut ni modifier le ledger ni voir les identités individuelles.

```

fair_value = reference_price
             + local_order_flow_adjustment
             - inventory * inventory_skew

half_spread = base_spread
             + volatility_component
             + latency_component
             + risk_component

```

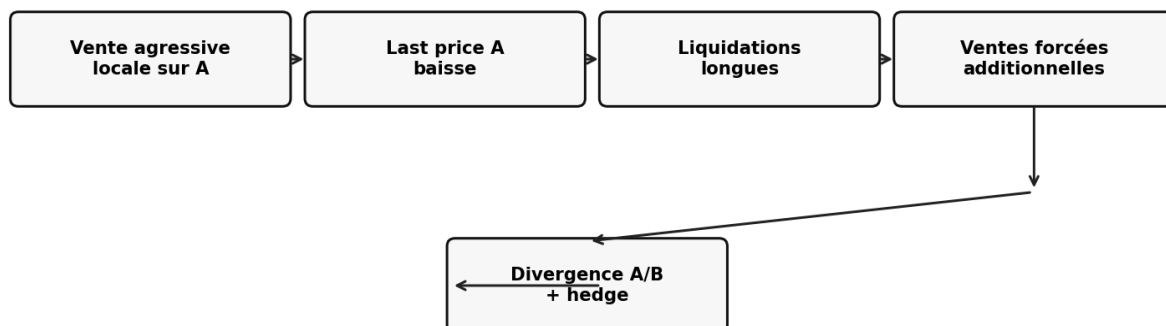
- Publier un bid et un ask sur les deux plateformes.
- Réduire la taille ou élargir le spread lorsque l'inventaire ou la volatilité augmente.
- Annuler les quotes périmées.
- Hedger l'exposition de A sur B, et inversement.
- Se retirer lorsque l'oracle devient stale ou que la connexion cross-exchange est perdue.
- Respecter un kill switch, une limite de perte et une limite de temps non hedgé.

Le panel god mode est strictement en lecture seule pour les positions. Il peut déclencher un kill switch global, mais ne peut jamais réécrire un trade ou un solde.

M7. Crash de A et arbitrage de liquidation

10 pts

Boucle de cascade simulée - Exchange A



La limite de 1 % est un paramètre du scénario local, pas une méthode de manipulation d'un marché réel.

Figure 3 - Boucle de cascade contrôlée sur l'échange A.

Le ScenarioController de A peut injecter un flux agressif synthétique capable de déplacer le prix local de 1 % au maximum. L'objectif est de tester le modèle de liquidation, pas d'enseigner une manipulation de marché réelle.

Conditions minimales

- Une majorité d'agents possède une position longue ou courte similaire.
- La distribution du levier crée des seuils de liquidation concentrés.
- Le carnet de A devient temporairement peu profond.
- A utilise son local last price comme mark price.
- B reste ancré sur l'oracle et conserve un mark price robuste.
- Les liquidations de A génèrent des ordres locaux supplémentaires.

Information privilégiée simulée

```
OpenInterestByLeverageBucket
LiquidationNotionalByPriceBand
AggregateMarginUtilization
InsuranceFundLevel
LocalDepth
MarketMakerInventoryBand
```

L'agent privilégié reçoit uniquement des données agrégées. Il ne reçoit ni identité, ni email, ni position nominative.

Agent d'arbitrage de liquidation

L'agent doit estimer si une divergence sur A est une opportunité de convergence ou le début d'une cascade. Il peut acheter sur A et vendre sur B, ou effectuer l'opération inverse.

```
expected_profit = expected_convergence
                  - fees
                  - expected_slippage
                  - inventory_cost
                  - liquidation_risk
                  - uncertainty_penalty
```

Métriques obligatoires

Métrique	Description
Net PnL	PnL réalisé + non réalisé - frais - funding
Maximum drawdown	Pire baisse depuis un plus haut de l'equity
Time unhedged	Durée cumulée au-delà de la limite de delta
Slippage	Écart entre prix de décision et VWAP de fill
Inventory peak	Exposition maximale par exchange et agrégée
Bankruptcy flag	L'agent a-t-il violé sa contrainte de solvabilité ?

Évaluation

La PnL brute ne suffit pas. Une stratégie qui gagne en prenant un risque illimité ou en restant non hedgée ne valide pas le jalon.

M8. Risk, slippage et VaR

8 pts

Contrôles pré-trade minimaux :

```
MAX_ORDER_SIZE
MAX_POSITION
MAX_NOTIONAL
MAX_LEVERAGE
MAX_DAILY_LOSS
ORDER_RATE_LIMIT
STALE_ORACLE
PRICE_BAND
INSUFFICIENT_MARGIN
KILL_SWITCH_ACTIVE
```

Slippage

```
execution_slippage_bps = side_sign
                        * (fill_vwap - arrival_price)
                        / arrival_price * 10000
```

Le calcul doit intégrer la profondeur locale, les exécutions partielles, les frais maker/taker et la latence.

VaR historique

Calculer une VaR historique 99 % à partir des rendements de l'oracle enregistré. Publier VaR_A, VaR_B, somme naïve et VaR nette cross-exchange.

```
net_btc_delta = delta_A + delta_B + option_delta
```

Les positions A et B sur le même sous-jacent doivent être dédoublées avant l'agrégation. Additionner aveuglément VaR_A et VaR_B est considéré incorrect lorsque les positions se couvrent.

M9. Options simples et volatilité

4 pts

Introduire une option européenne cash-settled sans construire une surface complète.

- Une seule maturité et un strike proche de l'ATM.
- Un call et un put ; taux sans risque fixé à zéro.
- Prix Black-Scholes, delta, valeur intrinsèque et valeur temps.
- Volatilité implicite obtenue par dichotomie.
- Un LongVolAgent achète le straddle et delta-hedge sur B.
- Un ShortVolAgent vend le straddle avec limites de perte et de gamma.

L'option peut être traitée par un OptionDeskActor à quote déterministe ; un second matching engine complet n'est pas demandé.

M10. Workflow KYC/AML pédagogique sur B

4 pts

Le but est d'apprendre la collecte minimale, le stockage et l'audit. Le projet ne doit jamais prétendre être juridiquement compliant.

Données synthétiques seulement

```
legal_name
date_of_birth
country
address
document_type
document_reference
document_expiry
source_of_funds_category
pep_self_declaration
terms_accepted_at
```

États

DRAFT -> SUBMITTED -> NEEDS_REVIEW -> APPROVED | REJECTED | EXPIRED

- A : email vérifié suffisant pour créer un compte simulé.
- B : trading bloqué tant que le dossier synthétique n'est pas APPROVED.
- Mini-panel administrateur : liste, détail, approbation, rejet motivé et audit trail.
- RBAC, chiffrement des champs sensibles, rétention configurable, aucune donnée sensible dans les logs.
- Aucun vrai document d'identité n'est accepté pendant l'évaluation.

M11. Tests, couverture et performances

6 pts

La couverture de lignes globale sur le code testable doit être supérieure ou égale à 50 %. Les fichiers générés et migrations peuvent être exclus de manière documentée.

Tests obligatoires

- Priorité prix-temps, partial fills, cancel avant fill, absence de double fill.
- Positions long -> short et short -> long, PnL et frais.
- Mark price A/B, funding, marge, liquidation partielle et totale.
- Distribution du levier et reproductibilité par seed.
- Oracle stale, reconnexion et replay.
- VaR avec et sans netting cross-exchange.
- RBAC KYC, approbation et rejet.
- Property-based test générant des séquences d'ordres et vérifiant les invariants.

Benchmarks

```
gateway -> matching
matching -> trade publication
trade -> position update
mark update -> liquidation decision
agent decision -> fill end-to-end
```

Publier count, min, mean, p50, p95, p99, p99.9 et max. Aucun seuil absolu n'est imposé ; une optimisation doit être mesurée avant/après avec résultats fonctionnels identiques.

Checklist de validation obligatoire

Contrôle	Preuve attendue
Oracle	Live, stale, disconnect et replay démontrés.
Matching	Invariants et idempotence couverts par tests.
Ledger	Réconciliation à zéro écart sur A et B.
Perpétuel	Funding, marge et liquidations vérifiés.
Population	200 agents minimum dans un scénario stable.
Crise	Cascade reproductible sur A, B reste robuste.
Arbitrage	Hedge mesuré, limites de risque respectées.
Qualité	Couverture $\geq 50\%$ et benchmark reproductible.
Sécurité	Aucune clé de trading ni donnée d'identité réelle.

6

Scénarios d'évaluation

Le projet est jugé sur des scénarios reproductibles, pas sur une démo improvisée.

6. Scénarios obligatoires

Scénario	Attendu
Marché calme	Spread stable, peu de liquidations, market maker proche de l'équilibre après frais.
News positive	Momentum achète, mean reversion vend progressivement, spreads s'élargissent.
Crowded long	Choc baissier local sur A, cascade possible, B reste proche de l'oracle.
Crowded short	Symétrique à la hausse.
Retrait de liquidité	Slippage augmente ; aucun invariant financier n'est violé.
Oracle stale	Nouvelles prises de risque bloquées ; quotes retirées selon la politique.
Déconnexion A/B	Le market maker réduit son risque et limite le temps non hedgé.
Insolvabilité	Compte négatif après slippage ; assurance simulée et ledger réconciliés.
Replay	Même seed, même hash fonctionnel.

Le scénario de crash doit produire au minimum les séries temporelles suivantes :

```
reference_price
local_mid_A / local_mid_B
mark_price_A / mark_price_B
open_interest_A / open_interest_B
liquidation_notional
insurance_fund
market_maker_inventory
arbitrage_agent_delta
aggregate_equity
```

Critères de réussite du scénario principal

- Le prix local A peut diverger sans que l'oracle soit modifié.
- La cascade sur A est causée par ses règles de mark et de marge, pas par un bug du matching.
- B ne liquide pas les mêmes comptes en l'absence d'un mouvement comparable du mark robuste.
- L'agent d'arbitrage explique ses décisions et reste dans ses limites de risque.
- Le ledger final de chaque exchange est réconciliable.

7

Évaluation

Cent points, des plafonds clairs et une soutenance technique.

7. Barème

Jalon	Partie	Points
M0	Infrastructure, protocole, replay	5
M1	Oracle Binance lecture seule	7
M2	Exchange A C++/Simplx	13
M3	Exchange B Python	8
M4	Perpétuel, marge, liquidations	15
M5	Agents et levier	12
M6	Market maker global	8
M7	Crash et arbitrage	10
M8	Risk, slippage, VaR	8
M9	Options simples	4
M10	KYC pédagogique	4
M11	Tests et performances	6
	TOTAL	100

Règles de plafonnement

Défaut	Conséquence
Le projet ne compile ou ne démarre pas	0 point
Un ordre réel peut être envoyé	0 point
Une donnée d'identité réelle est incluse	0 point
Un seul exchange fonctionne	Note maximale : 55
Pas de replay hors ligne	Note maximale : 60
Replay non déterministe	Note maximale : 70
Couverture < 50 %	Note maximale : 75
Mark price ou liquidations absents	Note maximale : 50
Ledger non réconciliable	Partie financière invalidée

Soutenance

L'équipe doit être capable de répondre sans lire son code :

- 1 Pourquoi le prix de Binance ne doit-il pas être injecté directement dans le carnet local ?
- 2 Pourquoi local last price et mark price ne doivent-ils pas toujours être identiques ?
- 3 Comment empêchez-vous un double fill après retry ?
- 4 Quelle différence existe entre marge initiale, marge de maintenance et equity ?
- 5 Pourquoi l'open interest seul ne suffit-il pas à prévoir précisément les liquidations ?
- 6 Comment le market maker mesure-t-il son risque non hedgé ?
- 7 Pourquoi une moyenne de latence est-elle insuffisante ?
- 8 Comment avez-vous agrégé la VaR entre A et B ?
- 9 Quelle optimisation avez-vous mesurée et non simplement supposée ?
- 10 Quelle hypothèse de votre modèle est la plus fragile ?

Déroulé recommandé de l'évaluation

Durée	Démonstration
5 min	Build propre, tests rapides et lancement du replay.
10 min	Architecture, protocole et séparation oracle / exchanges.
10 min	Scénario de crash A/B et analyse des liquidations.
10 min	Market maker, arbitrage, VaR et limites de risque.
10 min	Lecture de code, couverture, benchmark et questions.

8

Livraison

Le README doit permettre à un évaluateur de tout reproduire sans assistance.

8. Livrables

- 1 Code des deux exchanges et des agents.
- 2 Schéma canonique des événements et code généré ou validé.
- 3 Docker Compose ou scripts de lancement reproductibles.
- 4 Enregistrements d'oracle et scénarios de replay.
- 5 Tests, rapport de couverture et property-based tests.
- 6 Diagramme des acteurs Simplx et machine d'état des ordres.
- 7 Rapport du scénario de crash A/B.
- 8 Analyse du market maker et de l'agent d'arbitrage.
- 9 Rapport VaR, slippage, drawdown et assurance.
- 10 Rapport de benchmark C++/Python et avant/après optimisation.
- 11 Documentation du workflow KYC synthétique et de ses limites.
- 12 Une démonstration reproductible depuis une machine propre.

Arborescence recommandée

```
crashlab/  
|-- protocol/  
|-- oracle/  
|-- exchange-a-cpp/  
|-- exchange-b-python/  
|-- agents/  
|-- scenarios/  
|-- admin-web/  
|-- tests/  
|-- reports/  
|-- docker-compose.yml  
`-- Makefile
```

Definition of Done

Projet terminé

Deux exchanges fonctionnels + oracle public + replay déterministe + perpétuel + levier + liquidations + 200 agents + market maker + crash reproductible sur A + B robuste + arbitrage hedgé + VaR + options minimales + KYC synthétique + couverture $\geq 50\%$ + aucun ordre réel.

9

Bonus

Les bonus ne sont lus que lorsque la partie obligatoire est complète.

9. Bonus

Bonus	Contenu	Max
Dashboard	Carnets, positions, PnL, liquidations, open interest, assurance	5
PMS	Expositions par agent, stratégie, exchange et sous-jacent	4
TCA	Implementation shortfall, effective spread, adverse selection, fill ratio	5
Risk avancé	Expected Shortfall, stress tests, contribution à la VaR	5
Options avancées	Gamma, vega, theta, plusieurs strikes ou maturités	4
Agents avancés	Détection de régime, adaptation online ou LLM limité aux news	4

Un bonus ne donne aucun point si une partie obligatoire qu'il dépend de est absente ou incorrecte.

API publique étendue

Une API riche de création et supervision des agents est recommandée en bonus. L'API minimale d'ordres et de positions de l'échange B reste obligatoire.

A

Annexes techniques

Formules, événements et sources de référence.

A. Événements minimaux

```
NewOrder
CancelOrder
ReplaceOrder
OrderAccepted
OrderRejected
OrderCancelled
BookUpdate
Trade
PartialFill
PositionUpdate
MarginUpdate
FundingPayment
LiquidationRequested
LiquidationFill
ReferencePriceUpdate
MarkPriceUpdate
NewsEvent
RiskAlert
KillSwitchTriggered
```

Champs recommandés

Champ	Règle
event_id	Globalement unique et idempotent.
causation_id	Événement ayant provoqué celui-ci.
correlation_id	Chaîne complète d'une demande ou liquidation.
sequence	Monotone par producteur.
event_ts	Temps du scénario ou de l'échange.
received_ts	Temps de réception local pour les latences.
schema_version	Version explicite du protocole.

Open interest local

```
open_interest = sum(abs(position_i)) / 2
```

Heatmap de liquidation

```
liquidation_heatmap[price_band] =
    sum(notional_i that would liquidate
        if mark_price reaches price_band)
```

Le calcul doit être effectué uniquement à partir des positions locales de l'échange concerné.

B. Distribution du levier fournie

Monthly Weighted Average Effective Leverage XBTUSD			Percentile Effective Leverage XBTUSD		
Month	Long	Short	Percentile	Long	Short
2018.05	29	26	5%	2	2
2018.06	27	21	10%	3	3
2018.07	34	21	15%	4	4
2018.08	22	33	20%	4	5
2018.09	26	27	25%	5	6
2018.10	33	24	30%	7	7
2018.11	31	16	35%	8	8
2018.12	34	17	40%	10	9
2019.01	39	22	45%	11	10
2019.02	29	26	50%	13	13
2019.03	23	37	55%	16	16
2019.04	22	30	60%	20	19
			65%	25	22
			70%	30	27
			75%	40	34
			80%	54	47
			85%	79	59
			90%	93	84
			95%	95	92

Figure 4 - Image source fournie avec le sujet. La table de percentiles est utilisée comme distribution pédagogique.

Les moyennes mensuelles de la partie gauche ne sont pas utilisées pour tirer les agents. Seule la table de percentiles de droite sert à l'interpolation.

C. Références

[1] Binance Spot API - documentation générale et endpoints de données de marché publiques : developers.binance.com/en/docs/products/spot/rest-api

[2] Binance Spot WebSocket Streams - flux publics, notamment bookTicker et aggTrade : developers.binance.com/docs/binance-spot-api-docs/web-socket-streams

[3] Tredzone Simplx - framework C++ actor-based, tutoriels, événements, pipes et timers : github.com/Tredzone/simplx

[4] Distribution de levier XBTUSD - figure pédagogique fournie par le commanditaire du sujet.

Dernière règle

Le prix réel sert à ancrer la simulation. Il ne dispense jamais de construire un marché local cohérent.